

BeatDown: Self-Similarity As an Approach to Downbeat Detection

[DRAFT]

Recxa Lovelace
Independent Researcher
admin@rcxlv.llc

ABSTRACT

We propose BeatDown, an approach to offline downbeat detection that uses demixed self-similarity features to evaluate from a set of hypothetical downbeat alignments for a given audio sample. The system constructs a set of 16 downbeat "candidates," representing every possible downbeat alignment for that sample assuming consistent 4/4 time signature and a beatlist that is within 1 tempo scaling of ground truth (half, 1x, or 2x). For each candidate, a downbeat-stacked SSM matrix is constructed using 84-bin CQT features from the sample and all four of its demux stem channels (drums, bass, other, vocals). A gaussian checkerboard kernel is slid along the diagonal to extract novelty curves. 10 basic features are extracted from these curves such as sharpness and median, and a series of comparative metrics are calculated resulting in a 2000-d float vector representation of the candidate. A permutation-invariant neural network with candidate-set self-attention predicts the tempo and downbeat correctness of each candidate. Preliminary intradataset experiments yield approximately ~80% correct-candidate selection accuracy.

[Remaining work includes cross-dataset validation, and ablation experiments quantifying the contribution of demixed stems and multi-scale novelty features.]

1. INTRODUCTION

Automatic beat and downbeat detection have long been core problems in music information retrieval (MIR) due to the interplay of time-signature ambiguity, polyphonic overload, and cultural prescriptivism. Many early approaches relied on rule-based onset detection, hand-crafted tempo trackers, and heuristic "bar line" logic built on top of spectral features. These systems worked reasonably well in tightly constrained settings (e.g. dance music with steady drums), but tended to fail when presented with less predictable styles of music.

While recent approaches have made great strides in generalizing downbeat detection to novel or variant time signatures, the overall accuracy of SOTA libraries when applied to popular 4/4 western music remains surprisingly low given the availability of data and use cases. In practical pipelines, even low inaccuracy (10%-15%) can become costly down the line.

[Remaining introduction & related work paused until pending experimental data]

2. DATASET DESIGN

2.1 Source Datasets

There are a number of standard benchmark datasets for downbeat detection research, including the Ballroom, Beatles, Harmonix, and GTZAN datasets. For our preliminary evaluations, we decided to source our dataset from the popular rhythm game Osu!, using the recently published OSU2MIR toolkit. Osu! is a community-driven rhythm game in which players can create a timed "beatmap" to any song of their choosing. Thanks to the game's devoted charting community, Osu! has amassed a substantial corpus of downbeat-marked digital music, spanning a diverse range of genres and styles.

To download the beatmap files we used Batch Beatmap Downloader, a tool created by player nzbasic that allows users to mass download Osu! beatmaps based on various filters or parameters.

2.2 Filtering/Pruning

We restricted our download selection to include only ranked beatmaps with at least 200 favorites and a difficulty of "Insane" or higher. (As per OSU2MIR [formalize&cite]). This ensures that any sample beatmap has not only gone through the formal ranking process, but has also been informally screened the 200+ people who played and favorited it, maximizing the likelihood of a correct downbeat marking. This gave us a selection of approximately 26,000 beatmaps [verify exact number]. To prevent under-representation of the less popular genres, we capped the dataset at 1000 beatmaps per genre.

After downloading our selection, we further restricted the dataset to only include songs in persistent 4:4 meter. While our pipeline is compatible with any invariant time signature, the dataset only contained a small proportion of 3:4 samples, so expanding to a dual time-signature pipeline would likely worsen the model's accuracy. We do not restrict for tempo invariance - so long as the tempo change does not cause librosa to miss or hallucinate a beat, it will remain compatible with our pipeline.

3. PREPROCESSING PIPELINE

3.1 Shard Generation

For each beatmap in our dataset, we generate an empty JSON “shard” file. This shard will contain all necessary data required to train or evaluate with that sample. Using OSU2MIR, we extract a .txt file containing the sample’s beat times (in seconds) and beat positions, numbered 1-4. We copy the full beat-time list into the shard file under “beats”, and we also copy the subset of beat-times marked with beat position 1 into a “downbeats” list. We then run Librosa beat detection on the audio, leaving us with a ground-truth “beats” list, detected “beats” list, and a ground-truth “downbeats” list.

3.2 Candidate Construction

For each sample, we construct a set of downbeat-phase “candidates,” representing all possible alignments of the downbeat for a given beat-list and time signature. In common time 4/4, that would be a set of 4 candidates, one for each beat 1-4. However, librosa can sometimes track beats at the wrong tempo scale, detecting half-beats as beats or vice versa. For this reason, we expand the candidate set to also include augmented double-tempo and half-tempo beat lists. To construct a double-tempo beat list, we insert new beats at the half-way point between every original beat. To construct a half-tempo beat list, we remove every other beat. We do this twice, one list removing odd-numbered beats and the other removing even-numbered beats. This leaves us with 4 unique beat lists, each creating their own 4 downbeat alignment candidates, summing up to 16 downbeat candidates per beat detection. To make the most out of our dataset, we also construct candidate sets from the ground-truth beat list wherever librosa and ground-truth detect a different tempo scale. In total, we constructed [number around 5000 i think] candidate groups.

3.3 Marking Logic

To train our selection model, we assign each candidate two labels:

- A tempo class, relative to ground truth {quarter, half, 1x, 2x, 4x}
- A binary correctness flag, indicating whether or not the candidate’s downbeat phase aligns with ground-truth (irregardless of tempo class)

For tempo labeling, we define a set of BPM values at scale factors of ground-truth BPM value t : $\{0.25t, 0.5t, t, 2t, 4t\}$. We round the candidate’s BPM to the nearest value in the set, and mark it with the corresponding tempo label.

For correctness labeling, we define two correctness qualifiers:

- Cand→Truth (the percentage of candidate downbeats that lie within a 150 ms tolerance window of some ground-truth downbeat)
- Truth→Cand (the percentage of ground-truth downbeats that lie within a 150 ms tolerance window of some ground-truth downbeat)

When two beat-lists have a different tempo class, every beat from the lower-tempo list will land on some higher-tempo beat, but only half or a quarter of the higher-tempo beats will land on a lower-tempo beat. For this reason, we compare the lower-tempo list against the higher. Candidates above 1x tempo use Truth→Cand, and candidates below 1x tempo use Cand→Truth. For 1x tempo candidates, both of these qualifier values should end up nearly identical, so the choice is arbitrary. We use the average of both values.

Two perfectly aligned beat-lists would return a value of 1.0 from their respective qualifier. However, in practice we found that correct candidates often score in the range of 0.8-1.0 due to imprecisions in the data. We define a correctness threshold of 0.75 or higher.

3.4 Source Separation

Finally, we separate all sample audio into drums, bass, vocals, and other (a single stem containing all remaining instruments such as keys, guitar, etc) using Demucs v4 with default parameters. In section 5, we will use these stems in addition to the original mixture to execute a 5-channel feature extraction pipeline.

4. FEATURE EXTRACTION & NORMALIZATION

For each candidate in a candidate group, we run the following feature extraction pipeline:

4.1 Downbeat-synchronous CQT Extraction

We extract a beat-synchronous semitone spectrogram for each audio channel in the sample, synchronized to the candidate’s beat-list. The spectrogram is extracted with 84 CQT bins (constant-Q transform), and tuned using the librosa.estimate_tuning() method.

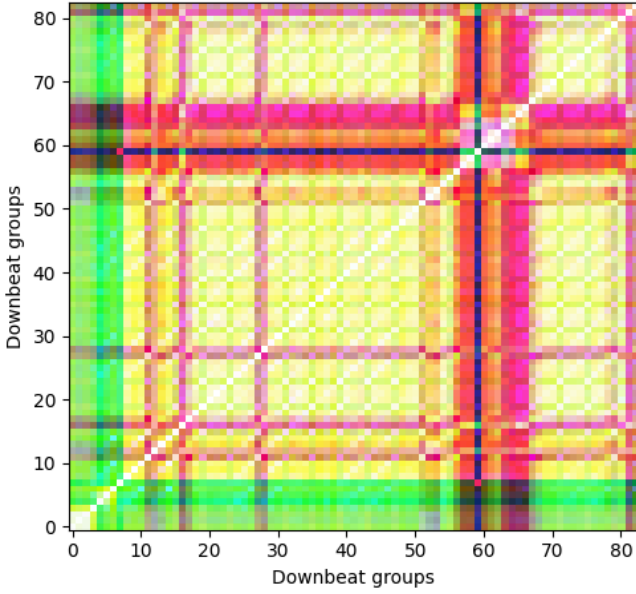
For each spectrogram, we group every set of four consecutive beat frames, beginning at the downbeat. We stack these frames in chronological order, creating a $(84 \times 4 = 336\text{-bin})$ downbeat-synchronous stacked semitone spectrogram. The group size of 4 assumes uninterrupted 4/4 common time – for 3/4, the group size would be 3, and so on.

4.2 Self-Similarity Matrix

For each downbeat-stacked spectrogram, we construct a self-similarity matrix (SSM) between all pairwise combinations of downbeat-synchronous frames, using cosine distance:

[cosine distance figure]

For visual insights into the structure of the song, we can map a selection of 3 audio channels to the RGB values of the SSM graph.



4.3 Novelty Curve

In our system's design, we proposed that the accuracy of a downbeat detection reflects in the sharpness or contrast of the detection's SSM. A standard way to extract these metrics is sliding a Gaussian checkerboard kernel along the main diagonal. We use this method, but expand it to include multiple kernel sizes and offset diagonals. We found that kernel sizes of 8 and 16 frames captured the most useful signal, and we set diagonal offsets at +4, -4, +8, and +16. At each end of the novelty curve, we apply a fade in/out proportional to the size of the kernel to prevent edge distortions.

4.4 Feature Extraction

We use a number of semi-arbitrary metrics derived from the novelty curves to assemble a float vector representing the candidate. We begin with a set of 10 basic metrics for each curve such as peak, mean, median, etc. We then calculate metrics for the overlap of different curves across stem, offset, and kernel-size dimensions. We are left with five 200-d vectors, one derived from each stem, concatenating to form one 1000-d raw candidate vector.

4.5 Within-Group Normalization

The feature extraction pipeline up to this point is unnormalized. Each individual metric has its own raw scale, some already bounded between -1 and 1 and others not. While the output range per-feature can vary wildly across different samples, we found that it groups much more tightly across candidates within a single set. For this reason, we normalize within-group. We use two separate normalization methods, rank and z (std). This allows us to preserve both scale and relative order. The final candidate vector is 2000-d.

5. MODEL ARCHITECTURE

5.1 Candidate-set input representation

As input, our model takes a set of normalized candidate vectors. By default, and for training and evaluation, we use the constructed 16-candidate format described above, amounting to 32,000 input nodes. However, our architecture is invariant to both the length and order of the candidate set.

5.2 Candidate encoder MLP

Each candidate runs through a shared encoder MLP, compressing the vector size from 2000 $\rightarrow$ 1024 $\rightarrow$ 512 $\rightarrow$ 256.

5.3 Set-wise self-Attention

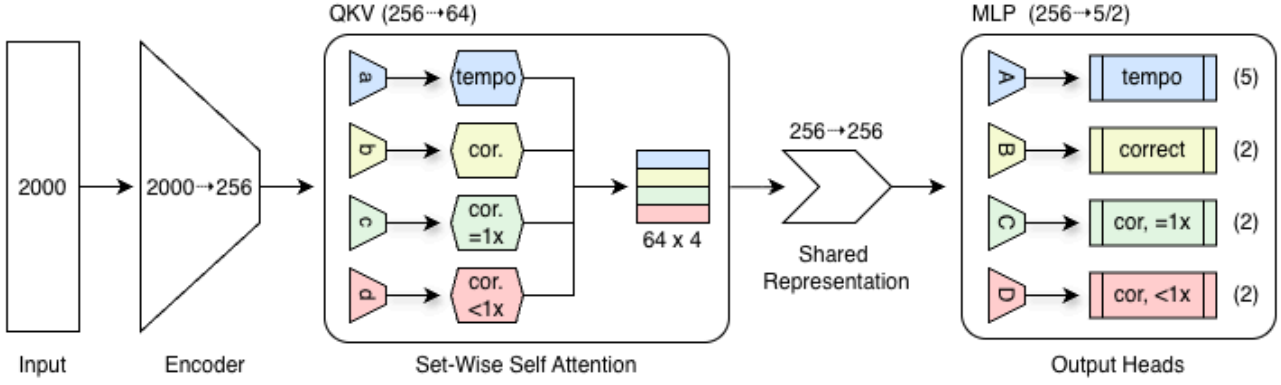
At the core of our model is a system for head-segregated, set-wise self attention. For each output head, the encoded 256-d candidate vector is sent through sets of shared head-specific QKV MLPs which compress the vector size from 256 $\rightarrow$ 512 $\rightarrow$ 256 $\rightarrow$ 64. These 64-d candidate vectors are then run through a head-specific self-attention matrix against all other candidates in the set. This allows the candidates to communicate, and allows each output head to enforce patterns specific to its rule without outsizing its influence over the other heads. The 64-d output of each matrix is concatenated back into one 256-d vector, and is then fed through a 256 $\rightarrow$ 256 $\rightarrow$ 256 shared representation MLP to integrate.

5.4 Output heads

The 256-d shared representation vector is fed into 4 shared head-specific MLPs, compressing the vector size from 256 $\rightarrow$ 256 $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ 16 $\rightarrow$ 5 or 2 (depending on the head).

Only 2 output heads are required to implement the most basic version of this model: tempo, and correctness. However, because of our definition of "correct" is technically independent of tempo, we are left with two metrics that might not inherently correlate the way we want them to. For this reason, we add a third output head: correct AND 1x-tempo ("true correct"). This gives our model a firmer handle on the task, but an issue remains. At tempo scales below 1x, even if downbeat alignment is "correct" by phase standards, the double/quadruple-wide "bar" groups that we end up with may or may not align with the musical structure of the song. If transitional points in the song land on a "true downbeat" that is somewhere in the middle of that 2/4-bar group we've predicted, it may produce features that are more resembling of an incorrectly-aligned candidate (blurry edges, strange lengths) than of a correctly-aligned candidate. For this reason, we add a final 4th head: correct AND $<1x$ -tempo. We deem this sufficient enough to give the model a fair chance.

For the tempo head, we have five output nodes: half, quarter, 1x, 2x, and 4x. For the remaining binary heads,



we use two nodes, representing true and false. We choose to do this rather than a singular binary output node to prevent forcing the model into a zero-sum paradigm. We posit that this allows the model to independently evaluate indicators of correctness and incorrectness without them needing to be inherently mutually exclusive, facilitating more nuance. However, this theory has not yet been evaluated.

To predict an individual candidate's output values, we softmax the output vector of each head, independently. To predict the "true correct" candidate, we softmax the set of "correct+1x TRUE" nodes between all 16 candidates, giving us the "truest correctest" candidate.

5.5 Activation function

For our activation function, we use a custom Sigmoid / ReLu blend. Each node is given two additional trainable parameters, one for its blend between the two functions (akin to a crossfader), and another for the "sharpness" (x-axis scaling) of the Sigmoid function.

6. TRAINING & EVALUATION

To train the model, we randomly partition the dataset into training and validation groups with a pre-determined ratio (e.g. 80/20%, 50/50%, etc). With our dataset size (~5000) training loss typically stabilized after around 50 epochs, which was not GPU-intensive and took around half an hour on a gaming PC and an hour on Colab's free CPU.

For robustness, we perform multiple runs with different splits (50/50, 70/30, 90/10), and different training parameters (lr 0.001/0.0001, 50/100/200 epochs).

In our preliminary test, we evaluate using 3 distinct metrics. Loss, accuracy, and output-node TP/TN/FP/FN.

7. RESULTS

In our 5 preliminary tests against custom Osu2MIR data, we achieved an average tempo accuracy of 0.832, and an average correctness accuracy of 0.812. Our average loss was 0.587. While TP/FP and TN/FP averages were close (~0.87 F, ~0.96 T), the gap was large enough that it allowed for high overall accuracy.

8. DISCUSSION

These preliminary results suggest that SSM-based "brute-force" candidate selection may be able to yield competitive downbeat alignment without operating directly in the audio domain. Given the comparative robustness of contemporary *beat-tracking* methods, our candidate-set system can be reliably expected to *include* the correct downbeat alignment hypothesis for the overwhelming majority of common-time, meter-invariant popular music.

However, as you may gather from the previous caveat, the current state of this work has several important limitations and flaws. Most critical of all is the lack of cross-validation – without direct comparison to an existing downbeat detection method, whether on our dataset or a standard dataset such as Harmonix, the true efficacy of this model is still conjecture. The reliability of our ground-truth data from OSU is also uncertain; unless manually verified by ear, there is no certainty that the users all applied consistent standards to their choice of tempo or even the beatmap's "formal metronome" at all.

Furthermore, even with competitive results on a 4/4 benchmark against existing systems, our architecture is fundamentally limited by its dependence on invariant 4/4 time signature. While the candidate construction system can be expanded to include any arbitrary time signature using the same mechanisms we used for 4/4, it would still rely on the song's time signature remaining constant throughout.

9. FUTURE WORK

9.1 Cross-Dataset Validation

As stated in the limitations section above, this work can not be properly evaluated in its context until it is cross-validated against multiple separate datasets and compared against existing systems. The rigidity of the current model's meter constraints makes that a challenge as only a subset of available data meets our criteria, so the external systems would have to be benchmarked manually on the specific subset of 4/4 music that we can use.

9.2 Un-Demixed Efficacy

Computationally, demixing is by far the largest bottleneck in this system's pipeline. Without demixing, a full single sample evaluation from execute to output would take 1-3 seconds on an average CPU. With demixing, it can take in the scale of minutes on certain hardware.

The effect that demixing has on our model's final efficacy has not been directly evaluated. Until tests have been run with the stem features omitted, it is unknown what role demixing had in the preliminary results we obtained. Regardless of the answer, there will be valuable insight gained by pursuing this test.

9.3 Time Signature Adaptation

As mentioned in section 9, our architecture is not incompatible with non-4/4 time signatures – however, each time signature needs to be accounted for manually. An additional output head can be added for time signature, and possibly some more for combinations a la "correct+1x." The main challenge is not the architecture, per se, it's the rigidity. Without a substantial amount of training data in non-4/4 time signatures, the model will likely struggle to translate 4/4 learning into other time signatures, and may end up worsening overall as a result of the noise. Until evaluated, it remains unknown.

9.4 Extensions

There are a handful of proposed extensions to this model that may potentially interface it better with the core proposal of this paper.

9.4.1 Visual ML

At the core of our paper is the proposal that the textural qualities of a downbeat-stacked SSM may indicate whether or not the downbeat-alignment is correct. While there are many benefits to the purely feature-based approach used so far, it is possibly that extending the candidate feature-vector to also include the output layer of a trainable visual CNN or MLP could have a significant impact on the model's grasp of the problem. SSMs are relatively low-dimensional, so computationally it's unlikely to be a bottleneck.

9.4.2 Within-Tempo Attention

While our model is input length and order-invariant, the 16-candidate version of this system does provide us with some information before the model has even run. Namely, we know what tempo-scaling group each candidate belongs to. From the model's perspective, these groups already show up extremely differently due to them halving or doubling the dimensions of the SSM and length of the curve in feature extraction. It is possible that a layer of within-tempo-group QKV self-attention *before* the initial shared encoder could allow the candidates to better place themselves in relation to one another.

9.4.3 Scenario Discernment

Similarly to how we know tempo-scaling groups ahead of time in the 16-candidate system, we also know exactly how *many* of each type of candidate there can be. For example, there can only be 3 different tempos represented in one group out of the 5 total possible tempos. Amongst those 3, they will always split into two groups of 4 and one group of 8. There are 4 possible scenarios for how many "correct" and "incorrect" candidates can exist across the tempo groups, depending on the relationship between the detection tempo and ground-truth tempo. It is possible that an extension can be appended to the model only in the case of 16-candidate evaluation, where the output layers (11-d) of each candidate is fed forwards into an additional self-attention layer before softmaxing. This layer would lead to two binary heads, one for "true tempo" and another for "true correct." The true correct nodes can then be softmaxed as is done in our current pipeline, yielding a singular answer.

10. CONCLUSION

While these results are preliminary, and there are many steps remaining to validate and deepen our findings, this work makes a compelling case for the viability of SSM novelty features as a method for downbeat detection. Even with cross-validated results, there are many architectural challenges preventing this system from generalizing to serve the same breadth of functionality as existing state of the art approaches. However, as it stands currently, this method may have some potential as a discriminating agent within more robust and flexible neural systems.